Assignment 2 - Genetic Programming for SBFL

Name : Rd Pradipta Gitaya Samiadji
Student ID : 20254099
Repository Link : https://github.com/Diptags/cs454-assignments

1. Overview

This assignment replicates the paper: a **Genetic Programming (GP)** engine, written from scratch, evolves a spectrum-based fault localisation (SBFL) formula `sbfl(ep, ef, np, nf)` that ranks the **faulty method** of 119 Defects4J bugs (Chart, Lang, Math, Time) as high as possible. The pipeline has four stages:

1. **Prepare** the raw coverage data into program spectra, once, and cache them (`data.py`).
2. **Evaluate** a formula: score every statement, aggregate to methods with max, and rank the faulty method (`evaluate.py`).
3. **Evolve** formulas with GP, using acc@1 and wef on training bugs as fitness (`gp.py`).
4. **Export** the single best formula, hard-coded, to `best.py`.

File	Contents
`data.py`	Converts the raw coverage (`bug_data/*-cov.pkl` + `*.json`) into statement spectra grouped by method; caches them in `spectra.pkl`
`evaluate.py`	Method-level ranking (max aggregation), tie-breaking, acc@1 and wef (overall and per project), human-designed baselines
`gp.py`	The GP engine: tree representation, initialisation, selection, crossover, mutation, fitness, train/validation split, multiple runs
`best.py`	The single best evolved formula, hard-coded (uses the required safe division)
`results.txt`, `runs/`	Results of all 18 GP runs and the final evaluation

Usage:

```
pip install -r requirements.txt
python data.py        # once: build spectra.pkl (~25 s)
python gp.py --runs 1 --pop 300 --gens 60 [--seed0 N]
             [--train-all] [--fitness acc|wef|log]
python evaluate.py    # baselines + best.py on all 119 bugs
```

The defaults are a population of 300, 60 generations and fitness `acc`. One GP run takes about 1.5–4 minutes; evaluating `best.py` on all 119 bugs takes less than a second. Every run is fully reproducible from its seed: re-running all 18 runs reproduced exactly the same formulas and numbers.

2. Data Preparation

2.1 Program spectrum

For every statement, I compute the four numbers of the spectrum from the coverage matrix (rows = statements, columns = tests) and the list of failing tests:

- `ef` / `ep`: number of **failing** / **passing** tests that execute the statement;
- `nf = F - ef` and `np = P - ep`, where F and P are the numbers of failing and passing tests.

The method of a statement is its identifier without the line number, e.g. `"pkg$Class#m(args):123".rsplit(":", 1)[0]`. All four counts are stored as plain Python `int`, because `sbfl()` must be called with ints. The spectra are computed **once** and cached, because the GP evaluates thousands of formulas on every bug.

2.2 A silent bug: sparse coverage data

The coverage DataFrames use a **sparse** boolean dtype. On sparse columns, `DataFrame.sum(axis=1)` behaves like a logical OR and returns True/False instead of a count. My first version therefore produced wrong spectra without any error (e.g. Lang-12 line 231 got `ef = 1` instead of 2). I found it by comparing against a spectrum computed by hand. The fix is to convert to a dense NumPy matrix first, and every statement is now checked against the invariants `ep + np = P` and `ef + nf = F`.

2.3 Dataset

Project	Bugs	Bugs with 1 failing test	Bugs with ≥ 2 failing tests	Methods (total)
Chart	19	12	7	10,773
Lang	34	22	12	6,140
Math	48	34	14	19,417
Time	18	9	9	24,919
All	**119**	**77**	**42**	**61,249**

Every faulty method appears in the coverage data, so no bug is impossible by construction.

3. Evaluation

3.1 Method-level ranking

- **Aggregation:** the score of a method is the **maximum** score of its statements.
- **Multiple faulty methods:** the best-ranked faulty method is used, as allowed by the assignment.
- **acc@1:** number of bugs whose faulty method is ranked **first**.
- **wef:** mean rank of the faulty method, i.e. how many methods a developer inspects before reaching the bug.

3.2 Tie-breaking

Many methods often receive the same score. Let *higher* be the number of non-faulty methods with a strictly higher score, and *tied* be the number with the same score. `evaluate.py` computes three ranks:

Scheme	Rank of the faulty method	Meaning	10 methods tied at the top
Pessimistic (conservative)	`higher + tied + 1`	the bug is inspected last in its tie group	10
Average	`higher + tied / 2 + 1`	tie group inspected in random order	5.5
Optimistic	`higher + 1`	the bug is inspected first in its tie group	1

The GP fitness and the headline numbers use pessimistic ties. With optimistic ties, a useless formula that gives every statement the same score would tie everything and rank every bug first, and the GP would learn to exploit that. For example, `Wong1 = ef` ranks **103/119** bugs first optimistically but only **2/119** pessimistically. I still report the other two schemes for comparison (Section 5.3). Note that for acc@1 the average and pessimistic schemes always agree: the average rank is exactly 1 only when nothing is tied with the faulty method.

3.3 Baselines and speed

- **Baselines:** Ochiai, Tarantula, Jaccard, Op2, Wong1 and AMPLE. They use `x / 0 = 0`, the usual convention in the literature. With the assignment's safe division (`x / 0 = 1`), statements executed by no test would get Ochiai = 1, the maximum score.
- **Speed:** many statements share the same spectrum, so each bug is compressed to its **unique** spectra (20,500 instead of about 250,000 statements). The formula is called once per unique spectrum with Python ints; the method maxima are then taken with `numpy.maximum.reduceat`. One evaluation on all 119 bugs takes about 12 ms.

4. GP Approach

No GP library is used. One run works as follows:

1. **Initialisation.** A random population is created with ramped half-and-half (depths 2–5, half *full* trees, half *grow* trees).
2. **Evaluation.** Every formula is compiled into a Python lambda and evaluated on the training bugs. Fitness is cached by the formula's source string, because crossover often recreates formulas that were already seen.
3. **Elitism.** The best 5 individuals are copied unchanged into the next generation.
4. **Variation.** The rest of the next generation is produced from parents chosen by tournament selection: crossover (80%), mutation (15%) or plain reproduction (5%).
5. **Repeat** for 60 generations, then return the best formula.

Aspect	Choice
Representation	Expression trees as nested tuples, e.g. `("div", "ef", ("sqrt", "ep"))` = `ef / sqrt(ep)`
Terminals	ep, `ef`, np, `nf`, constant `1`
Non-terminals	`+`, `-`, `*`, safe `/` (required definition), protected `sqrt(|x|)` (as in the reference paper)
Selection	Tournament of size 5; elitism of 5
Crossover	Subtree crossover: a random subtree of parent A is replaced by a random subtree of parent B
Mutation	Half subtree mutation (new random subtree of depth 1–3), half point mutation (change one node, keep the shape)
Bloat control	Maximum depth 8 (larger children are rejected); tree size is the last tie breaker in fitness
Population / generations	300 / 60
Fitness (minimised)	`(-acc@1, wef, size)` on the training bugs, pessimistic ties, compared lexicographically

4.1 Choosing the fitness function

I compared three fitness definitions in short pilot runs (population 200, 30–40 generations): wef first, mean $\log_2$(rank), and acc@1 first. On the 48 validation bugs, acc@1-first reached a mean validation acc@1 of 17.3 (3 runs), against 16.3 for log-rank (3 runs) and 17 for wef-first (1 run), while keeping wef competitive, so I chose acc@1 first.

4.2 Experimental protocol

1. **Generalisation check.** A stratified (per project) 60/40 split gives 71 training and 48 validation bugs. I ran 12 independent runs (seeds 100–105 and 200–205) and measured them on the validation bugs.
2. **Final runs.** The hidden test set comes from the same four projects, so I then trained on all 119 bugs (6 runs, seeds 300–302 and 400–402) and selected the run with the highest acc@1, then the lowest wef (seed 401).

5. Results

5.1 Best evolved formula

As evolved (`/` is the safe division and `sqrt` is `sqrt(|x|)`):

```
((sqrt((sqrt(((nf - ep) + sqrt(np))) * (ef * ef))) / (sqrt((sqrt((nf - ef)) + ((nf - ep) *
(ef - ep)))) * ((1 * (ef + nf)) / nf))) / sqrt((sqrt((nf - ef)) + ((((1 * 1) + nf) - ep) *
(ef - (1 + nf))))))
```

5.2 Why it works

The formula is easiest to understand through its behaviour. The table shows its value for P = 500 passing tests:

F	ef	ep = 0	1	2	5	10	50	200
1	1	2.18	2.15	1.23	0.45	0.20	0.05	0.02
3	3	2.57	9.47	23.77	0.71	0.16	0.02	0.00
3	2	0.85	1.45	1.43	0.38	0.15	0.03	0.01
3	1	0.38	0.85	1.45	0.17	0.04	0.00	0.00

- **One failing test (77 of 119 bugs).** The score decreases as `ep` grows: a statement executed by the failing test and by few passing tests is the most suspicious. For these bugs, every formula that increases with `ef` and decreases with `ep` gives the same ranking; Ochiai, Jaccard, Tarantula, Op2 and the GP formula all rank exactly 30 of the 77 first.
- **Several failing tests (42 bugs).** This is where formulas differ, and where the GP formula gains: **23/42** ranked first, against 16 for Ochiai, 15 for Jaccard and 13 for Op2. Statements executed by all failing tests (`nf = 0`) dominate those that miss some failing tests, and a large `ep` is penalised strongly (scores fall by orders of magnitude for `ep ≥ 10`). This combines the two properties of the maximal formulas from the lecture (Naish1/Op2, GP13): prioritise `ef`, then penalise `ep`.
- **Limitation.** Terms such as `(ef - ep)` and `(nf - ep)` create sharp peaks: with F = 3, a statement with `ep` = 1–2 scores higher than one with `ep` = 0. This is a pattern the GP learned from these projects, not a general rule, and it may not transfer to other programs.

5.3 Results on the 119 bugs

Formula	acc@1 pessimistic	acc@1 average	acc@1 optimistic	wef pessimistic	wef average	wef optimistic
GP (best.py)	**53**	**53**	**76**	**4.83**	**4.17**	**3.51**
Ochiai	46	46	70	5.09	4.48	3.87
Jaccard	45	45	70	5.22	4.60	3.97
Tarantula	42	42	71	5.53	4.85	4.18
Op2	43	43	65	15.27	14.67	14.07
AMPLE	42	42	64	11.57	10.97	10.36
Wong1	2	2	103	88.76	50.15	11.53

The GP formula is the best under every tie-breaking scheme, except for Wong1's optimistic acc@1, which is the "tie everything" effect from Section 3.2.

Average wef per project (pessimistic ties):

Project	Bugs	GP acc@1	GP wef	Ochiai acc@1	Ochiai wef
Chart	19	7	4.47	7	4.79
Lang	34	19	4.15	18	5.00
Math	48	19	5.12	15	5.10
Time	18	8	5.72	6	5.56
All	**119**	**53**	**4.83**	**46**	**5.09**

5.4 Robustness over 18 runs

A single run can be lucky, so I looked at all of them. On the same 48 validation bugs, Ochiai ranks 18 first (wef 5.79) and Jaccard 17.

Seed	Trained on	Validation acc@1 (of 48)	Validation wef	acc@1 on all 119	wef on all 119	Tree size
100	71 (split)	14	6.10	48	10.06	75
101	71 (split)	18	6.52	51	5.39	32
102	71 (split)	15	7.06	51	5.46	53
103	71 (split)	18	6.17	53	5.13	40
104	71 (split)	15	6.38	51	5.20	33
105	71 (split)	16	7.52	50	5.73	45
200	71 (split)	14	6.38	47	5.24	41
201	71 (split)	14	7.25	48	5.66	24
202	71 (split)	18	6.90	51	5.62	24
203	71 (split)	19	6.38	52	5.27	27
204	71 (split)	17	7.48	49	5.87	24
205	71 (split)	18	6.48	52	5.24	41
300	all 119	–	–	52	5.61	26
301	all 119	–	–	52	5.24	37
302	all 119	–	–	51	5.59	54
400	all 119	–	–	52	5.01	38
401 (best.py)	**all 119**	**–**	**–**	**53**	**4.83**	**54**
402	all 119	–	–	53	5.29	46

Validation acc@1 varies between 14 and 19 (mean 16.3), so the best runs generalise at least as well as Ochiai, but run-to-run variance is large. The original paper shows the same variance over its 30 runs. All final runs reach 51–53/119.

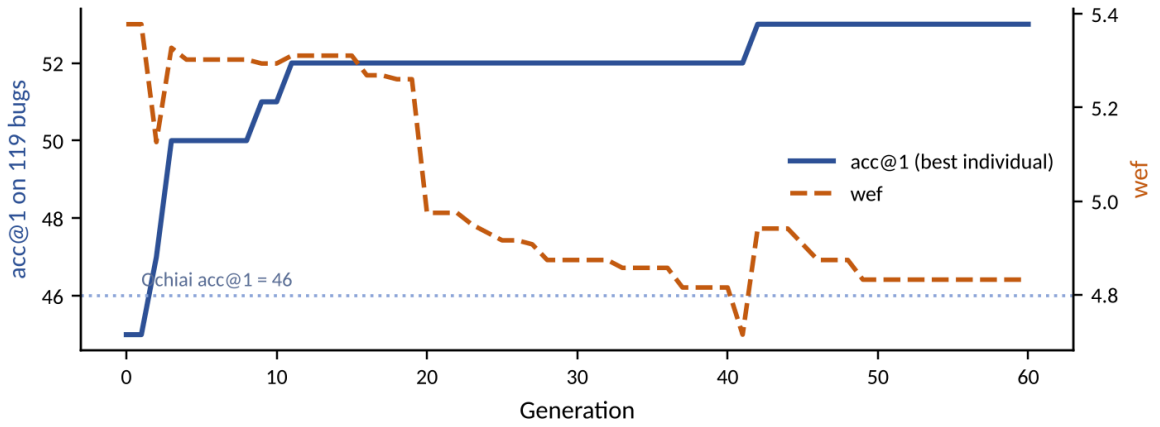


Figure 1. Convergence of the final run (seed 401): acc@1 and wef of the best individual per generation.

6. Discussion

- **Upper bound.** In 17 bugs, the faulty method has the same top spectrum as another method, so no formula with max aggregation can rank it strictly first. Only 88 of the 119 bugs are separable at all. Together with the 77 single-failing-test bugs, on which all reasonable formulas give 30 hits, about 50–55 strict top-1 hits is close to the practical ceiling of one SBFL formula on this data.
- **Tie-breaking matters more than the formula.** The same formulas rank 64–76 bugs first with optimistic ties but 42–53 with pessimistic ties. acc@1 values are only comparable under the same tie-breaking scheme.
- **Generalisation.** On held-out bugs, the evolved formulas are on par with or slightly better than Ochiai, not dramatically better. Selecting by validation results and keeping trees small helps against overfitting.
- **Limitations of SBFL itself** (from the lecture): omission faults have no statement to rank, and bugs with multiple faults blur the spectra; both are present in Defects4J.

7. What I Learnt So Far

- A program spectrum is just a 2×2 table per statement: who executed it, and did they pass or fail. Every SBFL formula is one way of turning those four numbers into a single score.
- Data processing can be wrong without any error. The sparse `sum()` problem only showed up because I compared the code against a small example I had computed by hand, so I now always check code against a hand-made example first.
- GP optimises exactly what the fitness measures. With optimistic tie-breaking, a formula that ties everything looks perfect, so the evaluation itself has to be designed so that it cannot be cheated.
- Most bugs here have only one failing test, and for those every sensible formula behaves the same. The real differences between formulas only appear when there are several failing tests.
- GP results vary a lot between runs, so a single run is not enough: I needed several seeds and a separate validation set to know whether a formula really generalises.